

Database Development Projektarbeit

Projektauftrag | Autor: Simon Erhardt

```

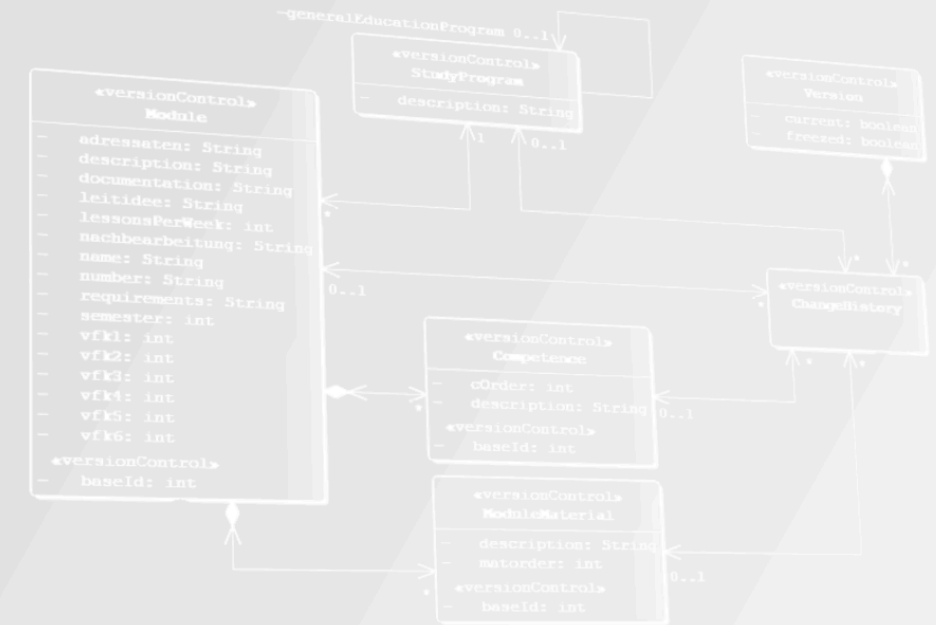
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144

    long second = (millis / 1000) % 60;
    long minute = (millis / (1000 * 60)) % 60;
    long hour = (millis / (1000 * 60 * 60)) % 24;
    return String.format("%02d:%02d:%02d", hour, minute, second);
}

private GameDurationDTO getShortestGameFromCloud() {
    GameDurationDTO gdt = new GameDurationDTO();
    if (TaaSBigScreen.getInstance().getTournament() != null) {
        try {
            gdt = ((TournamentService) ServiceLocator.getInstance().getService("tournamentService"))
                .getShortestGame();
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
    return gdt;
}

private GoalDifferenceDTO getBestGoalDifferenceFromCloud() {
    GoalDifferenceDTO gdf = new GoalDifferenceDTO();
    if (TaaSBigScreen.getInstance().getTournament() != null) {
        try {
            gdf = ((TournamentService) ServiceLocator.getInstance().getService("tournamentService"))
                .getBestGoalDifference();
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
    return gdf;
}

```



Eine eigene Anwendung entwickeln

Du verbindest dein Wissen aus DB2:

- ein tragfähiges Datenmodell entwerfen
- fachliche Regeln zuverlässig durchsetzen
- passende Datenzugriffe implementieren
- Verhalten testen und Performance untersuchen
- eigene Entscheidungen erklären

Ergebnis: eine lauffähige Spring-Boot-Anwendung mit PostgreSQL.

Rahmen und Eigenverantwortung

- Einzelarbeit
- 6 Projektlektionen inklusive Einführung
- 2–3 Stunden Hausarbeit
- zusätzlich 3 Lektionen Performance
- 5–10 Minuten individuelle Verteidigung vor der Klasse

Du teilst deine Arbeit selbst ein.

Abgabetermine stehen in Moodle. Die Schlusslektion organisiert der Dozent.

Einen überschaubaren Auftrag formulieren

Wähle ein Thema und definiere **5–7 überprüfbare Fachanforderungen**.

- Welchen Nutzen hat die Anwendung?
- Welche Vorgänge müssen funktionieren?
- Welche Regeln dürfen niemals verletzt werden?
- Welche Auswertungen werden gebraucht?

Alle vereinbarten Fachanforderungen und T1–T12 sind Pflicht.

API genügt: kein Frontend, keine Anmeldung, keine externen Dienste nötig.

Themenideen · Reservationen

Thema	Fachlicher Kern
Ferienwohnungen	Aufenthalte buchen und Verfügbarkeit prüfen
Kinoreservationen	Plätze vergeben und Auslastung auswerten
Carsharing	Fahrzeuge reservieren und Nutzung auswerten

Die Anforderungen und Abgrenzungen findest du im Auftragsblatt.

Themenideen · Weitere Möglichkeiten

Thema	Fachlicher Kern
Restaurantreservationen	Tische reservieren und Belegung prüfen
Pakettracking	Statusverlauf führen und Sendungen auswerten
Fitness-/Kursbuchungen	Teilnahmen buchen und Kapazität prüfen

Eigene Ideen sind möglich. Das bekannte Ticketsystem ist ausgeschlossen.

Dein Projektsteckbrief → Moodle

Reiche einen Steckbrief als PDF ein:

- 1. Thema:** Ausgangslage und Nutzen
- 2. Anforderungen:** 5–7 überprüfbare Fachanforderungen
- 3. Modellskizze:** Objekte, Beziehungen und Kardinalitäten
- 4. Kritische Geschäftsoperation:** Atomarität und Konfliktfall
- 5. Auswertungen:** fachliche Fragen an deine Daten

Datenmodell und Anwendung · T1–T3

Pflicht	Was du nachweist
Eigenes relationales Modell	Mindestens 3 zusammenhängende Tabellen, ER-Diagramm, Schlüssel und Constraints
Flyway-Migrationen	Aufbau ab leerer DB und nachfolgende Schemaänderung
Spring Boot und JPA	Fachfunktionen, Beziehungen, getrennte API-DTOs
Daten verwalten	Erfassen, lesen, ändern, löschen oder deaktivieren

Begründe deine Entscheidungen. Die API muss vorführbar sein.

Konsistenz unter Fehlern und Konflikten · T4–T5

Eine zusammengehörende Operation ist atomar.

Mehrere Schreibzugriffe gelingen gemeinsam oder werden zurückgenommen.

Gleichzeitige Änderungen bleiben fachlich korrekt.

Konflikte werden erkannt beziehungsweise verhindert und verständlich behandelt.

Nachweise: Erfolgsfall, Rollback nach Schreibzugriff und zwei unabhängige konkurrierende Zugriffe.

Gezielte Datenzugriffe · T6–T7

- JPQL-/DTO-Projektion für einen passenden Lesezugriff
- parametrisierte SQL-Auswertung über JDBC
- JOIN und Aggregation mit korrekten Ergebnissen
- eine tatsächlich verwendete Datenbank-View
- Filter, stabile Sortierung und Pagination in der Datenbank

Eine Auswertung darf mehrere Anforderungen erfüllen.

Verhalten mit PostgreSQL belegen · T8–T9

Automatisierte Integrationstests

Geschäftsregeln, Rollback, konkurrierende Änderungen und Abfrageergebnisse gegen echtes PostgreSQL prüfen.

Reproduzierbare grosse Datenmenge

Mindestens **100'000 Datensätze** in einer relevanten Vorgangstabelle samt zugehörigen Daten erzeugen.

Kleine Testfälle und grosse Performance-Daten dienen unterschiedlichen Nachweisen.

Performance untersuchen · T10–T11

Datenbankabfrage	ORM-Lesezugriff
Relevante Query auswählen	Beziehungen und Datenbedarf prüfen
EXPLAIN ANALYZE auswerten	SQL-Log oder Abfrageanzahl untersuchen
Index begründen und umsetzen	N+1 und unnötige Datenladung prüfen
Vorher und nachher vergleichen	Ladeentscheidung belegen

Gleiche Bedingungen, wiederholte Messungen, nachvollziehbare Erklärung.

Kein vorgeschriebener Beschleunigungsfaktor.

Cache und Aktualität · T12

Entscheide und begründe:

- Welche Ergebnisse werden gecacht?
- Welche Parameter bestimmen den Cache-Schlüssel?
- Was geschieht bei einer relevanten Datenänderung?

Zeige **ersten Zugriff**, **Cache-Treffer** und **Zugriff nach Änderung**.

Die Daten müssen deiner fachlichen Aktualitätsregel entsprechen.

Projektabgabe → GitHub

Dein Repository enthält:

- Quellcode, Migrationen und Testdatenerzeugung
- automatisierte Tests und API-Beispielaufrufe
- README mit Startanleitung, ER-Diagramm und Entscheidungen
- zugeordnete Nachweise für Fachanforderungen und T1–T12
- Performance-Messungen und Hilfsmitteldeklaration

Zugriffseinladung an simon.erhardt@hftm.ch

Zugriff sicherstellen; Abgabestand im README mit Git-Tag kennzeichnen.

Bewertung · Anwendung und Datenbank

Bereich	Punkte
Fachfunktionen, JPA und API	15
Datenmodell, Constraints und Migrationen	15
Transaktionen und Nebenläufigkeit	15
Query Design, JDBC, View und Pagination	15

60 von insgesamt 100 Punkten

Bewertung · Qualität und Verteidigung

Bereich	Punkte
Performance, Index und ORM-Ladeverhalten	15
Integrationstests und Reproduzierbarkeit	10
Cache und Aktualität	5
Individuelle Verteidigung	10

40 von insgesamt 100 Punkten

Verteidigung

Zeitraumen: 5–10 Minuten insgesamt

- **Präsentation: maximal 5 Minuten**
- **Rückfragen: maximal 5 Minuten**
- Zeige deine vorbereitete Anwendung kurz und verständlich.
- Begründe eine wichtige Datenbankentscheidung mit einem Nachweis.
- Beantworte eine fachliche Rückfrage selbstständig.

Bewertung: Demonstration 3, Begründung 4, Rückfrage 3 Punkte.

Die Organisation übernimmt der Dozent.

KI: HFTM-Richtlinie beachten

KI-Unterstützung ist im Rahmen der Richtlinie zulässig.
Du prüfst die Ergebnisse und verantwortest deine Arbeit.

Zur Abgabe gehören:

- Eigenständigkeitserklärung
- Hilfsmittelverzeichnis
- Prompt-Verzeichnis und nachvollziehbare Kennzeichnung

Keine vertraulichen oder personenbezogenen Daten in KI-Tools eingeben.

[HFTM: KI an der hftm einsetzen · Version 1.1, 02.02.2026](#)

Vorbereitung auf deinen Auftrag

Lies das Auftragsblatt und entwirf deinen Projektsteckbrief.

Prüfe deine Idee:

- Sind die Fachanforderungen klein genug und überprüfbar?
- Ermöglicht das Thema alle technischen Pflichtnachweise?
- Welche Geschäftsregel ist bei Fehlern oder Konflikten kritisch?
- Welche Auswertung eignet sich für die Performanceanalyse?

Kläre offene Fragen und den Umfang mit dem Dozenten.